Kleene: Relational Algebra for Recursive Language-Model Calls

A planner, a budget and a replay-gated playbook for agents that write SQL

Marcus Elwin
github.com/MarcusElwin/kleene

November 2026
Technical report, version 3 (draft; v1 of 3 October 2026, v2 of 9 October 2026)

Abstract

Recursive Language Models (RLMs) let a model treat its context as a variable in a REPL and call itself on slices of it. Every published harness gives the model Python for that REPL. We argue that for this job SQL is the better language, and we present Kleene, a Rust harness in which the model's only action is to write CallSQL: a PostgreSQL-flavoured dialect in which model calls are scalar and table functions, tools are table functions or `CALL` statements, delegation is a lateral table call that opens a child session, search is a recursive CTE with a beam limit, and the answer is a relation. Because the work is a query, three things follow that a tool-calling loop cannot offer. A *planner* sees the cost of every call predicate and applies eight rewrite rules (cheap-first, memo dedupe, batching, proxy cascades, semi-joins for `EXISTS`, beam-limited recursion, join ordering with branch-and-bound, volatility fences) with a cost model that `EXPLAIN` shows before any money is spent. *Budgets* in calls, tokens, dollars, depth and wall clock are query semantics: a statement whose estimate exceeds what remains is refused with its plan. *Learning* is rows in DuckDB: solved SQL becomes a playbook only after a replay evaluation with the memo bypassed wins on solves and cost, and every adoption is tied to the evaluation that admitted it. We describe the algebra, the system and a benchmark harness with eleven task packs run under three modes (learning, frozen, and a plain tool-calling agent on the same provider and budgets) on three models: Claude Opus 5.5, Claude Haiku 4.5 and GPT-6 Luna, every cell filled, plus three recursion-depth packs built to measure how deep child sessions nest. Opus saturates the four original packs, the coding pack and the four harder packs built after them (198 of 204 tasks), so on Opus the modes separate on cost only: the playbook cuts calls per task, the replay gate costs more than the playbook saves at twenty tasks a pack, and the one-call baseline is cheapest wherever the context fits in a prompt. Where it does not, the abstraction wins on the strongest model too: on a 30 000-token log Opus frozen solves 10/10 at a tenth of the cost of the plain agent's 5/10, and on the hard aggregation pack the plain agent on Opus costs four to six times the SQL modes and loses four tasks. Luna and Haiku separate on accuracy, and on the hard packs the gap between models (Opus 97%, Haiku 30 to 95%, Luna 0 to 62%) is larger than any gap between modes. The playbook's clearest wins are on the packs where the task is an aggregation or a draft: on rubric-graded memos learning beats frozen 18/20 to 8/20 on Luna and 11/20 to 8/20 on Haiku, and on long-context aggregation 20/20 to 15/20 on Luna at 1.3 calls a task against 34. On memo drafting the playbook hurt Opus (10 to 4), the one reversal. On the recursion-depth packs no model recursed along the tree: Opus solves all twenty tasks flat with recursive CTEs for $1.85, Haiku and Luna solve a fifth to a third, and only when the context no longer fits one call does Luna open child sessions five levels deep (1 634 of them), partitioning text rather than consolidating the tree. We report the numbers with their confounds (one run per cell, a shared memo in the first Haiku sweep, an untraced judge) and the harness fixes the runs forced, among them a cost model that learns token rates from the trace after a fixed rate had priced a cheap model out of recursing at all.

1 Introduction

Recursive Language Models [42] change what an agent harness hands the model. Instead of a prompt that holds the whole context, the root model sees only the task; the context lives as a variable in a REPL, and the model writes code that peeks at it, partitions it, and calls `llm_query` or `rlm_query` on slices. The strategies that emerge (peek, grep, partition-and-map, summarise, solve in code) are what one would write by hand, and the paper reports large gains on long-context aggregation at cost parity. The official harness and every reimplementation we know of give the model Python for that REPL. A reproduction study [39] found that recursion beyond depth one already degrades on current models, through format collapse and runaway sub-calls: the freedom of a general-purpose language is also the freedom to spend without a plan.

Kleene starts from a different premise. The REPL is a relational engine and the model writes SQL. Three properties of SQL make it a better fit for exactly this job.

1. **It is declarative.** The model says *what* relation it wants. The harness decides how many calls to make, in what order, concurrently or not, and on which model. That is precisely the freedom a planner needs and that an imperative loop denies it.
2. **It has an algebra with known complexity.** Select-project-join is NP-complete in combined complexity [6], full relational algebra is PSPACE-complete [38], and `WITH RECURSIVE` reaches Datalog and beyond [8], up to Turing completeness with `UNION ALL` [9]. Every statement the model writes can be labelled with the fragment it lives in, and join ordering over call predicates is a combinatorial search the planner deliberately confronts [14].
3. **The trace is a relation.** Calls, tokens, dollars and depths land in DuckDB tables [30]. The model, the harness and the person inspect execution with the language they drive it with.

Once the work is a query, a planner, budgets and learning fall out as database problems rather than prompt engineering. This report describes the resulting system and what it has measured so far. Our contributions are:

- CallSQL, a dialect in which model calls, tools, delegation and budgets are first-class (§2), and a *call algebra* that attaches a call kind and a cost signature to any relational operator that evaluates a call (§3).
- A planner with eight rewrite rules and a cost model that mirrors the executor's short-circuit semantics, exposed through `EXPLAIN` and used to refuse statements a budget cannot pay for (§3.1).
- A session loop with memoised calls, concurrent child sessions and a persistent store in which every learned object (playbook SQL, ratings, generator dials, refined prompt functions) is a versioned table adopted only after a replay evaluation (§4, §5).
- A benchmark harness with fourteen task packs, three modes that isolate the playbook and the SQL abstraction respectively, code and rubric oracles, and record-and-replay (§6); and the results of nine real-model sweeps on three models, reported with their confounds (§7), including the first measurement of how deep the models actually recurse (§7.9).

What this report does *not* claim is a benchmark win. The packs are small and mostly synthetic, every cell is one run, and Opus 5.5 solves every task of six of the eleven packs in every mode and all but a handful on four more. The numbers say where the abstraction pays for itself (aggregation over a context that does not fit, drafting against a rubric, a playbook on a stream of similar tasks) and where it does not (one-call lookups), and §10 lists what remains before a stronger claim can be made.

2 CallSQL: the language the model writes

Everything the model does in a session is one or more CallSQL statements in a single `sql` fence. The relational core is a PostgreSQL-flavoured subset: `SELECT` with CTEs and `WITH RECURSIVE`, inner and lateral joins, `EXISTS`, `IN`, scalar subqueries, grouping, ordering, set operations, `VALUES`, `CREATE TABLE ... AS`, `INSERT` and `DROP`, over the types `BOOLEAN`, `BIGINT`, `DOUBLE`, `TEXT`, `JSON` and `VECTOR`. Three-valued logic short-circuits `AND` and `OR` left to right, which the planner relies on. The core is tested differentially against DuckDB, recursive CTEs included (§7.1). On top of it sit five additions.

Context as a table. A run that is given a context starts with `ctx(ordinal, text)`, one row per paragraph. The root model sees the task, the catalog and the row count, not the text. It peeks with `SELECT ... LIMIT`, filters with `grep` or `LIKE`, partitions with `chunks`, and never has to hold the context in its prompt.

Model calls as functions. The scalar builtins `llm(prompt)` (with optional alias and effort arguments), `llm_bool(prompt)` and `llm_json(prompt, schema)` each cost one call per distinct argument tuple. `expand(text, n)` is a table builtin that returns n successors. Prompt-defined functions give a call a name, a return type, a model tier and a volatility:

```
CREATE FUNCTION mentions_hours(note TEXT) RETURNS BOOLEAN
  AS PROMPT 'Does this note record hours worked? {note}'
  MODEL 'proxy' BATCH 20 IMMUTABLE;

CREATE FUNCTION is_valid(c TEXT) RETURNS BOOLEAN
  AS PROMPT 'Is {c} a valid answer to the task?'
  MODEL 'worker' PROXY plausibility THRESHOLDS (0.2, 0.8);
```

`MODEL` picks a routing alias (`root`, `worker`, `proxy`, `judge`, or any alias the router knows). A prompt function is batched: the executor answers up to n distinct tuples per call, 20 unless `BATCH n` says otherwise (`BATCH 1` opts out), and the planner prices $\lceil \text{rows}/n \rceil$ calls. `PROXY` declares a cheap scorer the planner may cascade the predicate through. Bodies may also be SQL (`AS SQL (SELECT ...)`) or a shell command (`AS SHELL`); volatility defaults to `IMMUTABLE` for prompts, `STABLE` for SQL and `VOLATILE` for shell.

Tools. Pure tools are table functions usable in `FROM`: `files`, `lines`, `read`, `grep`, `chunks`, `env`, `git_log`, `git_diff`, `git_blame`. Side effects run as statements: `CALL shell(cmd)`, `CALL write_file(path, text)`, `CALL patch(path, old, new)`, `CALL web_search(q)`, and so on. A tool's output relation is rendered back to the model like any other result. Roles restrict which tools a session may use, and a run's capability profile (§4) narrows them further.

Delegation. `rlm(question [, context])` in a `CROSS JOIN LATERAL` opens one child session per input row at depth $d+1$ and returns `(answer, detail)`. `CREATE AGENT` declares a role with a model, an effort, a tool list, a budget and a prompt; `spawn('role', task)` runs it. A child's catalog is restricted to its role's tools and its budget is the parent's remaining slice intersected with the role's budget.

```
CREATE TABLE parts AS
  SELECT (ordinal / 20) AS part, string_agg(text, '\n') AS chunk
  FROM ctx GROUP BY 1;

CREATE TABLE answers AS
  SELECT p.part, r.answer
  FROM parts p CROSS JOIN LATERAL rlm(
```

```
    'Total hours per project in these notes, as project: hours', p.chunk) r;
```

Finishing, budgets and plans. `FINAL FROM (query)` ends the session with that relation as the answer. `SET` adjusts effort, model, memo and the budget dimensions within a session. `EXPLAIN` prints the call plan (§3.1); `EXPLAIN ANALYZE` runs it and prints actuals beside estimates. The harness runs the planner on every statement before execution, and a statement whose estimate exceeds the remaining budget is returned to the model as a refusal carrying the plan rather than executed.

The opening example of the repository shows the shape the design is built for:

```
SELECT candidate
FROM possibilities
WHERE VERIFY(candidate)
  AND NOT EXISTS (
    SELECT 1 FROM counterexamples ce
    WHERE REFUTE(candidate, ce.text));
```

`VERIFY` is one call per distinct candidate. The `NOT EXISTS` is an anti-semi-join that stops on the first refuting counterexample: the counterexample-guided loop of CEGIS and CEGAR [7, 34] written relationally. `EXPLAIN` says how many calls that is before they are made.

3 The call algebra

The ordinary operators are selection σ, projection π, join $\bowtie$, union $\cup$, difference, aggregation γ and the least fixpoint μ for recursive CTEs. KLEENE adds a *call kind* to any operator that evaluates a call expression (Table 1). The cost of an operator is the number of calls it implies times the per-alias token and dollar factors of the function it calls; selectivity of a boolean call predicate is a prior until the predicate has been observed, after which the sampled pass rate is used.

Operator	Meaning	Cost signature
λ_f (map-call)	scalar call f per distinct input tuple	$\lvert\delta(R)\rvert \times \text{tokens}(f)$
κ_g (expand-call)	table call g per input tuple (`LATERAL g(...)`)	$\lvert R\rvert \times \text{tokens}(g)$; output = branching factor
ρ_d (recurse)	`rlm` / `spawn` at depth d	the child's whole plan
σ_{llm}, $\bowtie_{\text{llm}}$	filter or join whose predicate contains a call	inherits λ_f; $\lvert R \times S\rvert$ calls for a join unless rewritten

Table 1: Call kinds. δ is duplicate elimination; identical prompts cost one call.

Every statement is also labelled with the fragment it lives in: CQ for select-project-join, FO for the full relational algebra, REC when a recursive term is present. The badge is a reminder that a recursive call plan is a program, not a lookup.

3.1 Rewrite rules and the cost model

The planner in `kleene-algebra` runs rules over the annotated plan until nothing changes. All eight are standard query-rewrite shapes applied to call costs.

1. **Cheap first.** $\sigma_{\text{pure}} \circ \sigma_{\text{llm}} \equiv \sigma_{\text{llm}} \circ \sigma_{\text{pure}}$: pure conjuncts run before call conjuncts, in filters and in join conditions. Because estimates mirror the executor's left-to-right short-circuit, the saving is visible in the numbers rather than assumed.

2. **Dedupe then map.** $\lambda_f(R) \equiv \lambda_f(\delta(R)) \bowtie R$, backed by a persistent memo keyed on model and prompt fingerprint and shared across sessions and runs.
3. **Batch.** λ_f over n tuples of a prompt function becomes one call with n numbered items, priced $\lceil \text{rows}/n \rceil$. Answers come back as `{"item": i, "answer": ...}` and are matched by item; the missing subset is re-asked once as a batch, and only then one call per tuple. The first runs lacked the re-ask and batched only on request, and paid for it (§7.5).
4. **Cascade.** $\sigma_{\text{oracle}}(R)$ with a declared proxy becomes score $\geq \tau_{\text{hi}} \vee$ (score $\geq \tau_{\text{lo}} \wedge$ oracle), applied when the priced cascade is cheaper; the shape is LOTUS's [29], with thresholds declared rather than calibrated (§10).
5. **Semi-join.** `[NOT] EXISTS` with a call predicate becomes a semi or anti join that stops at the first (counter)example instead of materialising the cross product.
6. **Beam-limited recursion.** Inside μ, a trailing `ORDER BY ... LIMIT k` in the recursive term keeps the best k new rows of each round: breadth-first search with a model successor function becomes beam search of width k, and the planner simulates the fixpoint round by round to price it.
7. **Join ordering.** Dynamic programming over relation subsets with branch-and-bound, in the tradition of Selinger et al. [33] and Moerkotte and Neumann [28]: each split is priced with the predicates that first become applicable there, pure conjuncts narrowing the pairs and call conjuncts paying one call per surviving pair.
8. **Volatility fences.** Rules 1–7 move operators only across `IMMUTABLE` and `STABLE` operators. A `VOLATILE` operator (a shell command, a file write) is a fence: nothing crosses it, it is never deduplicated, and its input order is preserved. The planner trusts the label, so a side effect is never marked anything else.

Estimates use the store's row counts, sampled selectivities, per-alias cost factors and pricing. The per-token rates are learned: once three priced calls are in the store's `trace_calls`, the cost model prices tokens at the rate those calls actually paid (output to input fixed at the rate cards' five to one, cache reads at a tenth) rather than at a built-in default, a change forced by the recursion runs of §7.9, where a fixed rate twenty times too high for a cheap model refused every delegation. `EXPLAIN` prints the plan as a tree with per-node rows, calls, tokens and dollars, the call kinds, the fences, the fragment badge, the rules that fired, the join-order plan space, the alternatives it priced and the rates it is using.

Budgets as semantics. A `Budget` has calls, tokens, dollars, depth and wall clock. A child session receives the parent's remaining slice intersected with its role's budget, and its spending rolls up. Exceeding a budget cancels the statement, not the session, and returns a budget row to the model; a statement whose *estimate* already exceeds the remainder is refused before it runs. Depth defaults to a cap of two, following the reproduction study's finding that deeper recursion degrades on current models [39]; it is a dial, not a constant: a benchmark pack states the depth its tasks need and the harness raises the cap to it, and `-max-depth` overrides both. A child gets half of its parent's *remaining* budget, so under a bounded budget the slice a session can spend falls geometrically with depth.

4 System

KLEENE is a Rust workspace of eleven crates with a shared interface crate (`kleene-core`: values, schemas, the catalog, budgets, call kinds) that every other crate depends on. Figure 1 shows the path of one turn.

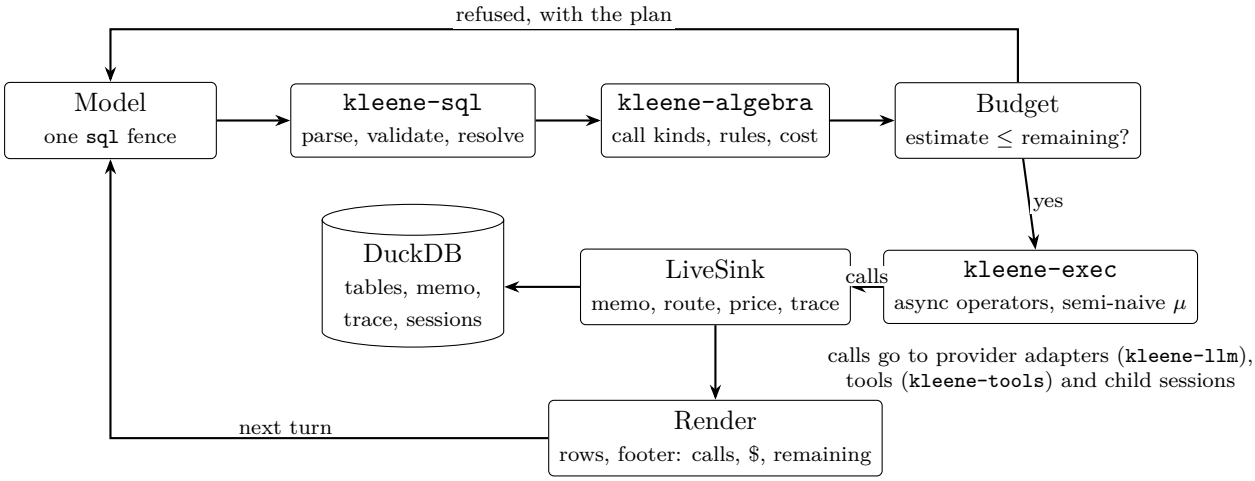


Figure 1: One turn of a session. The harness sends a cached system prefix (rules, catalog, budget) plus the transcript; the model replies with CallSQL; the statement is parsed, annotated, priced, checked against the budget, executed through a sink that memoises, routes and traces every call, and rendered back. The loop repeats until `FINAL`.

Execution. Operators are pull-based async streams of small columnar batches. Recursion runs by semi-naive evaluation [3]: each round joins only the previous round's delta against the recursive term, so calls in that term run only on new rows, and a round is one awaitable unit the UI can show as a frontier size. Model calls are concurrent under a per-session semaphore with a provider rate limiter; ordered results are preserved.

The sink. Every scalar, table, tool or child call passes through a `LiveSink` that looks the call up in the memo, and on a miss routes it to a provider alias, prices it at the router's list rates, charges the budget, stores the response in the memo and emits a trace event. Identical prompts are therefore free the second time, across sessions and runs; this is also how a recorded run replays offline.

Sessions and delegation. Child sessions are the same loop one level deeper with a role, a budget slice and their own table namespace; a child's `FINAL` returns to the parent as a row. Children of one lateral call run concurrently. Session state (tables, transcript, budget) lives in the store, so killing the process and running `kleene resume` continues where it stopped; checkpointing is a property of the storage, not a feature.

Model layer. Adapters speak the Anthropic Messages and OpenAI-compatible wire formats directly over HTTPS, plus an Open Responses adapter behind a feature flag; there are no vendor SDKs. A router maps aliases to models and prices, and a `ReplayProvider` serves recorded fixtures so the test suite and the benchmark harness can run without a key. The Anthropic adapter puts a cache breakpoint on the last message of every multi-turn request, not only on the system prefix, so a root session reads its transcript from cache from the third turn on; the plain baseline does the same. Both adapters rewrite model-written `llm_json` schemas into the subset their provider's structured-output mode accepts (closed objects with every property required; a non-object root wrapped and unwrapped; typeless properties completed). Every one of these was forced by a benchmark run (§7).

Store and trace. DuckDB holds session tables, the memo, the trace (`trace_sessions`, `trace_statements`, call events), the continual loop's tables and the benchmark rows. `kleene`

`trace` runs plain DuckDB SQL over all of it, and the terminal UI's `/trace` does the same from the prompt.

Memory as tables. A long session is not compacted into a summary; it is folded into tables the model can query. Once more than sixteen turns have run, the earlier ones move out of the prompt into `turns(run, n, sql, result)` and the last eight stay in full, so a fact from turn three is a `SELECT` away rather than a paraphrase. Tasks typed in the terminal UI belong to a *conversation*: every run records its task and its answer in `conversations(conversation, run, n, role, text, ts)`, a follow-up run's task message shows the last four messages inline and points at the table for the rest, and `/new` starts another conversation. `memory(key, value)` is a plain table the model may `INSERT INTO` to keep a fact across runs. This is the RLM premise applied to the harness's own state: the context is a variable the model reads with queries, and the place where other harnesses summarise is a table here.

Capability profiles. Four flags on `run`, `repl`, `tui` and `attach` say which tools a run's sessions may use: `-read-only` withholds every `VOLATILE` tool (shell, file writes, network, writable MCP tools), `-no-shell` and `-no-web` withhold those two, and `-tools a,b,c` keeps only the names listed; a `[profile]` section of the configuration file sets a default. A withheld tool is removed from the catalog, so the SQL frontend does not resolve it and the prompt does not list it; the prompt names the withheld tools instead, so the model does not plan around `CALL shell` and get refused. Children inherit the profile: a `CREATE AGENT ... TOOLS` role is intersected with it, never widened. The `-check` command still runs whatever the profile says, because it is the user's command, not the model's.

Processes. `kleene run`, `repl`, `explain`, `learn` and `bench` run the harness in one process. The terminal UI talks to an engine daemon over a Unix socket (a named pipe on Windows) with a JSONL protocol with cursors, so a reconnecting client resumes from where it left off and `kleene attach` watches the same events headless. Since v0.4.0 the engine, the REPL, the UI and the providers run natively on Windows from a release zip; the `shell` tool and the benchmark packs run commands through `sh`, so they need one on the path, and text in and out is UTF-8 whatever the console code page (a context file in the console's code page is refused with the fix named, never loaded as mojibake), which a field report in Japanese was the first to test. The UI is one scrolling stream: the model's reply streams in, each fence runs as it lands, every statement shows its rows and its cost, child sessions nest under the turn that opened them.

5 Learning as tables

The continual loop (`kleene learn`) is itself a query over a `tasks` table: pick the next pending task by curriculum policy, run a session, judge `FINAL` with the task's oracle, record outcome and cost, repeat. Two things keep it alive: generators that keep the table non-empty, and learning that changes how the next task is attempted.

Tasks and difficulty. Generators (`sat3`, `graph`, `puzzle`, `corpus`, `repo`, `statements`, `contracts`, `logbook`, `memo`) produce tasks deterministically from a hardness dial in $[0, 1]$ and a seed, each with a code oracle; the design follows Reasoning Gym [35] and the random 3-SAT phase transition [27]. Every task and every solver configuration carries a rating moved by a Bradley–Terry model [5] over solve and fail outcomes: a task that beats strong configurations is hard, a configuration that beats hard tasks is strong. The curriculum samples the pending task nearest even odds for the current solver, steps a generator's dial up when its solve rate exceeds 0.7 and down below 0.3, in the spirit of the learnability target of Absolute Zero Reasoner [43].

What is learned. Nothing here updates weights. Each learned object is a table with a version and a provenance pointer:

- the **playbook**: SQL that solved a task, keyed by task kind, rendered into the cached prefix as examples for similar tasks and queryable by the model;
- **refined prompt functions** (`learn refine`): new versions of `VERIFY`-style functions in a ledger with before/after snapshots and rollback;
- the **cost model**: sampled selectivities of boolean call predicates (today per session; §10) and the per-token rates learned from the priced calls in the trace (§3.1);
- **ratings** of tasks and solver configurations, and each generator's **dial**.

The replay gate. A candidate playbook entry is adopted only after a replay evaluation: the harness re-runs a sample of earlier tasks of the same kind in two arms, with and without the candidate, with the memo bypassed, and adopts the entry only if the arm with it wins on solves and costs no more. The sample is three tasks by default, so a gate costs six task runs per solved task. Every adoption is tied in `playbook_evals` to the evaluation that admitted it, and `learn playbook` lists reverts, so a regression can be traced to the entry that caused it. The base system prompt is immutable and learned content is appended in marked sections. §7.2 shows that at twenty tasks a pack the gate spends far more than the playbook saves; it is the price of never adopting an entry on faith.

6 Benchmark methodology

The harness's claim is specific: a model writing CallSQL over a relational engine solves the same tasks as a tool-calling agent for fewer calls and dollars, and the harness gets better over a stream of tasks because solved SQL is kept. The benchmark harness (`kleene bench`) is built to test exactly that, so every run records three things per task: was it solved, how many model calls it took, and what it cost.

6.1 Packs

A pack is a directory with a `pack.json`: tasks with text, optional context, setup commands, a workspace source, an oracle and a difficulty prior. Table 2 lists the fourteen shipped packs: the seven of the first two versions of this report, four *hard* packs built in October after every model solved the original aggregation, finance and legal packs in every mode, and three recursion-depth packs (§7.9). The synthetic packs are frozen output of the loop's generators, so two runs see the same tasks in the same order and `bench build` regenerates them byte for byte; the long-context, hard and recursion packs are stored lazily as (generator, dial, seed) triples and regenerated on load. The hard packs are new generators rather than a higher dial, because the old ones at dial 1.0 stay one-shot readable: `tracker` mixes time units, opens action items that are closed later by id alone and amends entries with corrections; `ledger` is a quarterly statement by segment over six years with restatements and a discontinued segment; `agreement` is a 350-section contract with a near miss for every planted category and amendments that delete, add or touch a clause. `terminal-hard` is hand-written: eight workspaces of hundreds of files or thousands of rows, generated at setup, with a hidden grader that recomputes the answer from the files. The coding pack is hand-written: four small Python projects, each a three-step *episode* in which the second and third steps run in the workspace the previous step left, and a step's setup installs the reference solution for the previous step when its hidden tests fail, so each step is measured on its own work. The external datasets these packs imitate (OOLONG [4], Terminal-Bench [26], FinanceBench [15], CUAD [13], Harvey LAB [12], SWE-bench [16]) are not redistributed; a

Harvey LAB checkout imports through `bench import-lab` with LAB's own evaluator as the scorer of record.

6.2 Modes

Every pack runs under one of three modes, and two comparisons fall out of them.

`learning` KLEENE: the model writes CallSQL, the playbook is shown, candidates are recorded and adopted through the gate, ratings move.

`frozen` KLEENE with the same model, tools and budgets, but no playbook shown, no candidate recorded, no rating moved. The control for learning.

`plain` a tool-calling agent on the same provider, tools and budgets, in the ReAct shape [41]: the tools go to the model as native tool definitions, every tool-use block of a reply runs and its result comes back as a tool result, and a `final` tool carries the answer. The baseline for cost parity, as in the RLM paper's framing: same model, same spend, only the abstraction differs.

`learning` against `frozen` isolates the playbook; `frozen` against `plain` isolates the SQL abstraction. Two caveats matter for reading the results. Both KLEENE modes read the same store, so a frozen run on a store that has seen a learning run (or the reverse) is served from the memo for every prompt already sent; a clean comparison needs a fresh store per pack and mode, which every sweep after the first Haiku one does (§7.3). And the first Haiku sweep's plain agent parsed one JSON action per reply from text; it was replaced by the native tool-calling agent above before the later sweeps, and the coding pack's tasks now carry a `check` that refuses a `FINAL` while the project's tests fail.

6.3 What is recorded

`bench run` writes one row per task to `evals`: run, pack, mode, position in the stream, task, solved, the oracle's reason, calls, tokens, dollars, deepest child, turns and wall clock. `-budget-dollars` caps every task in every mode; a capped task is recorded as unsolved with `budget_exhausted` as its reason, so a cap is a result, not a gap. `bench report` aggregates by pack and mode; `bench csv` exports the rows; `bench curve` prints the rolling solve rate of one run; `bench plot` draws five standalone SVGs (learning curve, accuracy at cost parity, calls against difficulty, planner estimate against actual, plan-space size against query shape) without a plotting library. The replay evaluations a learning run pays for are recorded in `playbook_evals` with their own dollars and are *not* in `evals`; the true cost of a learning run is the sum. `bench run -record` saves every model reply as a fixture and `-replay` serves it back, so a run reproduces offline; `-resume` continues a run that a provider error stopped.

7 Results

Pack	n	What the model gets and must produce	Oracle	Stands in for
`terminal`	6	a workspace and a shell instruction; files left behind	`shell`	Terminal-Bench
`oolong-like`	20	dated meeting notes as `ctx`; hours per project	`exact`	OOLONG aggregation
`finance-synthetic`	20	a synthetic income statement; one number	`number`	FinanceBench
`legal-synthetic`	20	a synthetic contract; the clause categories present	`exact`	CUAD / LAB
`logbook-hard`	30	a 30 000-token work log with corrections and distractors; one of five aggregate questions	`exact`	OOLONG at full size
`memo-rubric`	20	a 40-section agreement with an amendment and a rejected proposal; a short memo	`judge`	LAB drafting
`coding`	12	a Python project with visible tests; implement, extend	`shell`	SWE-bench episodes

7.1 Deterministic measurements

Table 3 is the part of the evidence that needs no model: the test suite runs it in CI with scripted providers, code oracles and replay fixtures.

Property	Test	Result
Relational core equals DuckDB	property-based differential suite	passes, recursive CTEs included
Join ordering, two call predicates over three relations	estimate and actual calls against a counting sink	as written 12 000 estimated calls, chosen order 60; actual $\geq 10\times$ fewer with identical rows
Plan-space growth	seven-way join	665 280 bushy orders priced through 1 623 subset splits (309 pruned)
Cascade	proxy at $0.1\times$ cost, thresholds $(0.2, 0.8)$	100 proxy + 60 oracle calls beat 100 oracle calls by estimate; on data the oracle is asked only for the band (5 of 20)
Beam recursion	`expand` $\times 4$, three rounds	20 calls with beam 5 against > 100 unbounded
Continual loop	scripted solver over generated puzzles	tasks solved, playbook v1 adopted after a replay eval, ratings and dial move, restart resumes
Terminal-Bench-style pack	shell tasks from SQL	tasks solved, oracles run in fresh workspaces
Plain-agent baseline	same provider, native tool calls	tool call then final; the control for cost parity

Table 3: What the deterministic suite establishes.

7.2 Claude Opus 5.5 on the first four packs

The four original packs ran once each under the three modes on 27 September 2026 with the default routing: `root` on `claude-opus-5-5` at high effort, `worker` and `judge` on `claude-sonnet-5`, `proxy` on `claude-haiku-4-5`, priced at the router's list rates. Table 4 gives the per-pack rows; Figure 2 the plots `bench plot` drew from the same store. `dollars` is the pack total, calls and tokens are per task, and the learning column excludes the gate.

Pack	Mode	Solved	Calls / task	Tokens / task	Dollars
terminal (6)	frozen	6/6	2.17	10 551	0.07
	learning	6/6	1.33	6 742	0.04
	plain	6/6	2.50	2 395	0.03
oolong-like (20)	frozen	20/20	4.90	30 778	1.51
	learning	20/20	3.85	41 991	1.92
	plain	20/20	1.00	3 084	0.39
finance-synthetic (20)	frozen	20/20	3.45	42 177	0.83
	learning	20/20	2.40	29 999	0.63
	plain	20/20	1.00	1 244	0.07
legal-synthetic (20)	frozen	20/20	12.65	62 452	1.69
	learning	20/20	10.60	58 315	2.61
	plain	20/20	1.00	1 475	0.11

Table 4: Claude Opus 5.5, 27 September 2026, one run per cell. Gate replays (not included): \$1.25 terminal, \$10.70 oolong, \$5.46 finance, \$16.63 legal.

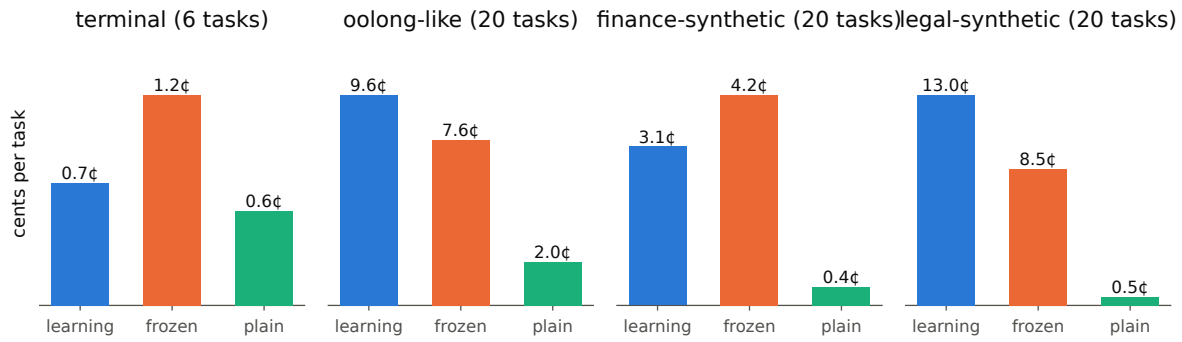


Figure 2: The Opus 5.5 run: mean cents per task by mode, one panel per pack. Every task was solved in every mode, so cost is the only axis. The plain agent is cheapest wherever the context fits one prompt; learning is below frozen on every pack, by run-to-run variance on terminal and finance where nothing was adopted.

Accuracy is saturated. Opus 5.5 solves every task of every pack in every mode. The packs were sized for a weaker model at dial 0.5; they separate nothing on accuracy, so what follows is about calls and dollars.

The playbook cuts calls per task. Learning against frozen: terminal 2.17 to 1.33, oolong 4.90 to 3.85, finance 3.45 to 2.40, legal 12.65 to 10.60. Only two packs adopted anything, though: oolong (16 of 20 candidates) and legal (5 of 21). On terminal and finance every candidate was rejected because its replay cost more than the baseline's, so those learning runs are frozen runs with ratings on, and their call reduction is run-to-run variance, not learning. On oolong the second half of the learning run took 3.9 calls per task where the first half took 3.8: most of the gain arrived with the first adopted entries. On legal the halves are 13.7 and 7.5, the curve the design predicts.

The gate is the cost. Each candidate is replayed on three earlier tasks in both arms, six task runs per solved task, and those replays cost $1.25, $10.70, $5.46 and $16.63 on the four packs against $0.04, $1.92, $0.63 and $2.61 for the counted runs. At twenty tasks a pack the gate spends far more than the playbook saves. It earns its keep only over a long stream, and a cheaper gate (a smaller sample, a proxy model, a cached baseline arm) is the obvious next change.

The plain agent wins on cost here. Every context in these packs fits in one prompt (under 4 000 characters), so the baseline answers in one call with no tool use: $0.39 against $1.51 on oolong, $0.07 against $0.83 on finance, $0.11 against $1.69 on legal. The SQL abstraction pays for a catalog, a planner and a transcript of rendered results on every turn, and at this scale that is pure overhead. The claim of §1 is about work that does not fit in one call; these packs do not test it, and the numbers say so.

Per-clause calls are cheap and visible. On legal the model classified clause by clause with a prompt function on the `worker` alias, up to 26 calls a task at about $0.10: the λ_f map-call doing what it is for, many small calls on a cheap model, priced before they run.

Costs drift up within a run. Per-task dollars on oolong learning rose from $0.084 in the first half to $0.107 in the second while calls stayed flat. Two things grow the prompt: the adopted playbook entries, and, as it turned out, the scratch tables every session created in the shared

store, which the catalog listed to every later session (287 such tables after these runs). The second was a bug: it also changed the prompt fingerprint from task to task, so a recorded run could not be replayed from its own fixtures. Bench sessions now drop their tables when they finish.

7.3 Claude Haiku 4.5 on the harder packs

The harder packs ran on 1 October 2026 with every solver alias on `claude-haiku-4-5-20251001` and the `judge` alias on `claude-sonnet-5-5`. The sweep was stopped before it finished; Table 5 is what completed. Not in the table: `logbook-hard` in any mode (a frozen task took about 500 calls and over twenty minutes at Haiku's pace, so the pack was capped at ten tasks and then stopped before the first row landed), and `coding` in plain mode (below). Gate replays cost $1.43 on memo-rubric (11 candidates, 3 adopted) and $1.04 on coding (2 candidates, 1 adopted); the counted runs cost $3.36 in all.

Pack	Mode	Solved	Calls / task	Tokens / task	Dollars
coding (12)	frozen	2/12	4.58	48 375	0.87
	learning	2/12	2.67	29 910	0.47
memo-rubric (20)	frozen	8/20	19.95	49 043	1.30
	learning	11/20	3.80	23 041	0.37
	plain	8/20	3.15	10 764	0.35

Table 5: Claude Haiku 4.5 solvers with a Sonnet 5.5 judge, 1 October 2026, one run per cell; the sweep was stopped before `logbook-hard` and the coding plain run produced rows. The learning rows share the frozen rows' memo (see text).

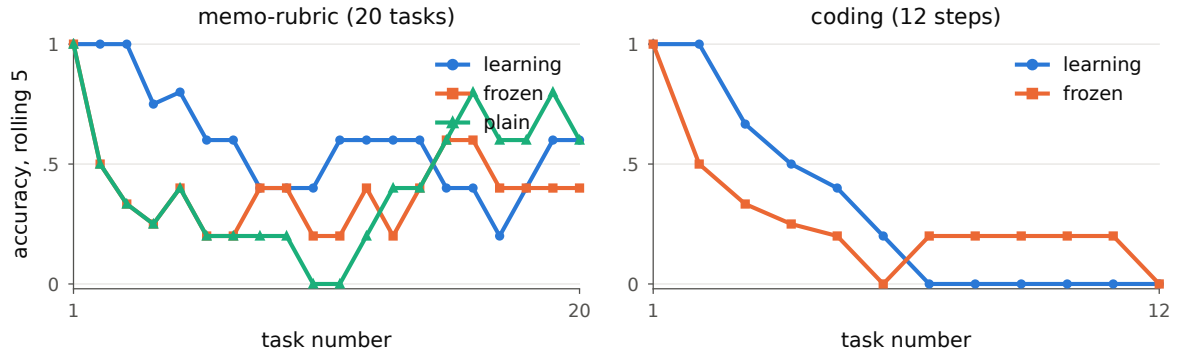


Figure 3: The 1 October Haiku 4.5 run: accuracy over the last five tasks by position in the stream. On memo-rubric the learning line ends above frozen and plain; on coding both KLEENE modes solve only the first step of each episode and fall to zero.

These packs separate on accuracy. Where Opus 5.5 solved every task of the first four packs, Haiku solves 17% of the coding steps and 40 to 55% of the memos. The coding steps it solves are the first step of an episode; it solved no "extend" or "fix the bug report" step in either KLEENE mode.

Learning beat frozen on memo-rubric, on both axes. 11/20 against 8/20, at 3.8 calls a task against 19.95 and $0.37 against $1.30. Two effects are mixed in. Three playbook entries were adopted, and the second half of the learning run solved 5 of 10 where the frozen run's first

half solved 3 of 10. But most of the frozen run's calls are a failure mode the playbook happens to steer around: Haiku writes `llm_json` output schemas the API rejects (`additionalProperties` unset, `minItems` above 1, a bare `{"hours": "number"}`), and the statement retries until the call budget is gone; five frozen tasks took 22 to 141 calls each. The adapter now rewrites a model-written schema into the accepted subset before the request; these runs predate it.

Learning and frozen share the memo. The learning run came second in the same store, so a prompt identical to one the frozen run sent was answered from the memo for free. Two coding steps in the learning run finished with zero calls: they replayed the frozen run's failed answers. The learning column's calls and dollars are therefore not an independent measurement; every later sweep used a fresh store per pack and mode.

The plain baseline needs a protocol the model keeps. The agent asks for one JSON action per reply. Haiku answers with five to ten actions and invented tool output between them; the first extractor took the span from the first brace to the last, which never parsed, so on every coding task the model saw only a parse error and gave up after two turns (0/12 at $0.45). The extractor now takes the first balanced object and runs it. With that, Haiku worked through the first coding task for all 30 turns without solving it ($1.11) and the rerun was stopped before a row landed. On memo-rubric, where the task fits in one call, the plain agent matches frozen on accuracy at a quarter of the cost, and learning beats it by three tasks for the same money.

The judge wobbles. Sonnet 5.5's first lines included "PASS (with caveat)", "PASS (provisional)" and "PASS or FAIL?", all counted as passes by the first parser. It now requires a first line that says PASS and does not mention FAIL. The memo frozen and learning runs predate that change; the plain run does not.

Two provider facts. Haiku rejects the `effort` parameter the adapter sent along with the rest of the model family, so under the default routing every `proxy` call failed until the adapter learned to omit it; and `claude-sonnet-5-5` was served on 1 October where it was not on 27 September.

7.4 GPT-6 Luna on the first seven packs

All seven packs ran on 5 October 2026 with every solver alias on `gpt-6-luna` (OpenAI chat completions, default reasoning effort) and the `judge` alias on `claude-sonnet-5-5`. Luna is priced at $0.10 per million input tokens and $0.50 per million output, a tenth of Haiku and a fortieth of Opus. Each pack and mode ran in its own store, so the learning columns are independent of the frozen ones. `logbook-hard` was capped at ten tasks after the Haiku experience; its learning run was stopped after three tasks, when the second task alone took 1 705 calls and ran out of turns. Table 6 gives the rows; Figure 4 the learning curves. The counted runs cost $0.74 of Luna in all; the gate's replays (84 candidates, 17 adopted) another $0.31; the Sonnet judge's calls are not traced, roughly a dollar at most.

The original packs separate on accuracy with Luna. Opus solved every task of these packs; Luna does not, so they measure something now. The one-call plain agent is perfect on finance and legal, where the context fits and the question is one lookup, and weakest on oolong-like (14/20), where aggregation over the whole context is the task. Learning on oolong-like solved 20/20 at 1.3 calls a task against frozen's 15/20 at 34: the adopted playbook entry carries the aggregation query, and the rest is a lookup.

Pack	Mode	Solved	Calls / task	Tokens / task	Dollars
terminal (6)	frozen	6/6	1.50	4 128	0.00
	learning	5/6	1.50	4 219	0.00
	plain	6/6	2.33	1 584	0.00
oolong-like (20)	frozen	15/20	33.95	19 230	0.04
	learning	20/20	1.30	4 516	0.00
	plain	14/20	1.00	2 031	0.01
finance-synthetic (20)	frozen	14/20	4.15	11 789	0.01
	learning	15/20	3.45	12 081	0.01
	plain	20/20	1.00	950	0.00
legal-synthetic (20)	frozen	18/20	15.65	23 973	0.03
	learning	15/20	4.25	15 893	0.01
	plain	20/20	1.00	1 004	0.00
coding (12)	frozen	11/12	12.08	55 286	0.02
	learning	10/12	8.17	42 094	0.02
	plain	9/12	10.25	27 860	0.03
memo-rubric (20)	frozen	8/20	13.60	36 571	0.03
	learning	18/20	6.10	27 683	0.02
	plain	11/20	1.00	1 978	0.01
logbook-hard (10)	frozen	7/10	247.30	180 736	0.24
logbook-hard (3 of 10)	learning	1/3	686.67	746 259	0.19
logbook-hard (10)	plain	3/10	2.30	97 000	0.07

Table 6: GPT-6 Luna solvers with a Sonnet 5.5 judge, 5 October 2026, one run per cell, one store per cell. Dollars below half a cent print as 0.00.

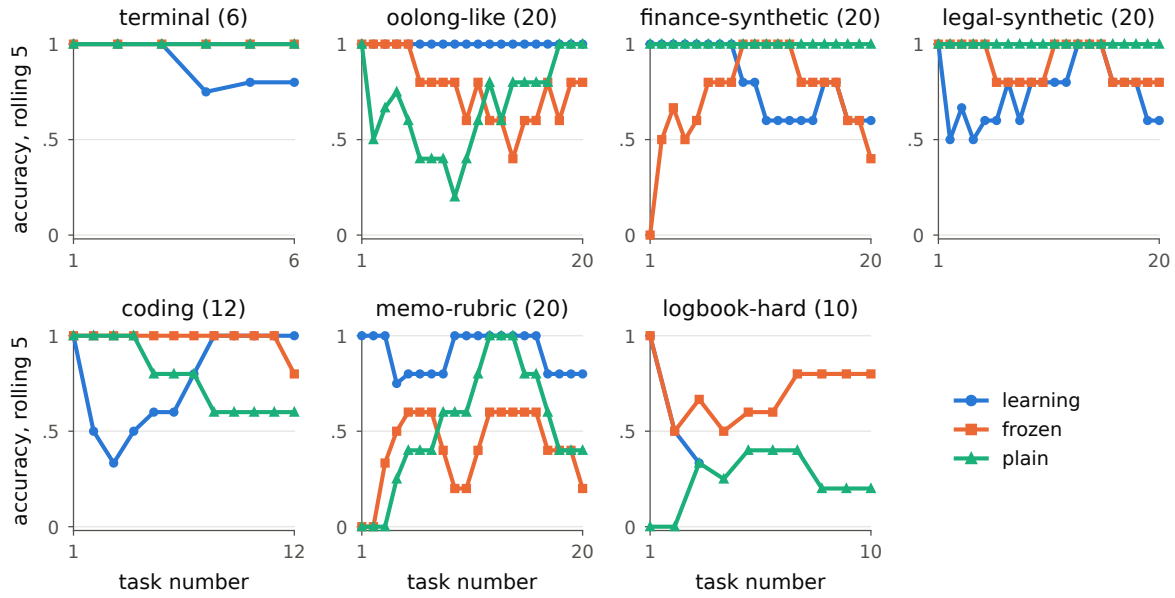


Figure 4: The Luna run: accuracy over the last five tasks by position in the stream, one panel per pack. On memo-rubric the learning line sits at or above 0.8 from the start where frozen hovers near 0.4; on oolong-like learning is flat at 1 where frozen and plain wander; on logbook-hard frozen climbs to 0.8 while plain stays near 0.2 (the learning run stopped after three tasks).

Luna solves what Haiku could not, for a tenth of the money. Eleven of the twelve coding steps frozen against Haiku's two, including every "extend" and "fix the bug report" step but one; seven of ten logbook tasks frozen where Haiku had not finished one. Each coding step cost about a fifth of a cent.

Learning beat frozen on memo-rubric again, now cleanly. 18/20 against 8/20 at 6.1 calls a task against 13.6, in separate stores, so no memo replay hides in the learning column. Four memo entries were adopted out of 18 candidates; both halves of the learning run solved 9 of 10, so the gap to frozen is there from the start and is mostly the shorter route the playbook gives. On coding, learning cost fewer calls (8.2 against 12.1) and lost one task.

Plain is the cheap mode where the context fits, and no further. On memo-rubric the one-call agent solves 11/20 for half a cent; on logbook-hard, whose contexts are around 30 000 tokens, it gets 3/10, and on coding 9/12 with a tool loop. Frozen beats it on every one of the harder packs on accuracy.

Logbook-hard is where the calls go. Frozen tasks took 113 to 719 calls each, nearly all of them `worker` calls from batched `llm_json` extraction over the log: Luna sometimes answers a batch of 20 with the wrong number of elements, and the harness then retried one call per row. Learning was worse, not better: the one task it finished past the first took 1 705 calls and no `FINAL`. This per-row fallback was the next thing fixed (§3.1, rule 3).

Three provider facts. OpenAI rejects `max_tokens` for GPT-5 and GPT-6 models (the adapter sends `max_completion_tokens`); strict structured outputs want an object root with every property required and `additionalProperties: false`, so a non-object schema is wrapped and unwrapped; and a model-written schema with a typeless property or a partial `required` list is rejected outright, so the adapter completes it. Before those fixes a Luna run could not make a single `llm_json` call.

7.5 Claude Haiku 4.5, the remaining cells

The cells the 1 October sweep left empty ran on 6 October 2026 with the same routing, one fresh store per pack and mode, after two adapter changes: the conversation is cached turn by turn, and the structured-output schema drops `maxItems`, which the API rejects and every batched `llm_json` call used to set (Table 7). The counted runs cost $20.07, of which `logbook-hard` took $15.66; the gate's replays (62 candidates, 22 adopted) another $5.64.

On the original packs Haiku is close to Luna, at ten times the price. Learning beats frozen on oolong-like (19 against 14 at 2 calls a task against 50) and legal (17 against 14 at 10 against 44), as it did for Luna; the one-call plain agent is perfect on finance and oolong-like.

Logbook-hard is a worker-call storm. The frozen run made 79 324 `worker` calls of about 37 tokens each, $4.30 of its $5.13: Haiku writes a scalar `llm()` per row over the 30 000-token log, and when it does batch it answers with the wrong number of elements two times in three, so the harness fell back to one call per row. Two tasks took 5 000 and 8 500 calls. The root session itself was cheap (121 calls, $0.83, 1.2M tokens read from cache against 36 000 billed in full). This is the fallback the Luna run flagged, now with a price on it; the fix landed the same day: prompt functions batch 20 rows per call unless told otherwise, and a short batch answer is re-asked by item instead of one call per row.

Pack	Mode	Solved	Calls / task	Tokens / task	Dollars
terminal (6)	frozen	6/6	3.00	11 190	0.07
	learning	6/6	2.17	8 159	0.05
	plain	6/6	2.50	7 286	0.05
oolong-like (20)	frozen	14/20	50.40	53 217	0.75
	learning	19/20	1.95	9 115	0.20
	plain	20/20	4.15	24 932	0.49
finance-synthetic (20)	frozen	20/20	7.25	30 274	0.43
	learning	19/20	7.60	18 963	0.29
	plain	20/20	1.65	5 375	0.13
legal-synthetic (20)	frozen	14/20	43.50	48 594	0.72
	learning	17/20	9.95	16 541	0.29
	plain	15/20	1.85	6 227	0.16
coding (12)	plain	4/12	28.58	218 019	0.78
logbook-hard (10)	frozen	1/10	1 702.10	476 213	5.13
	learning	1/10	1 694.40	591 443	5.95
	plain	2/10	22.20	1 365 343	4.58

Table 7: Claude Haiku 4.5 solvers with a Sonnet 5.5 judge, 6 October 2026, one run per cell, one store per cell. The coding plain row uses the native tool-calling agent; the coding frozen and learning rows and the memo-rubric rows are those of Table 5.

Caching works where the prompt is long enough. Haiku's minimum cacheable prompt is 4 096 tokens, so terminal and the per-row worker calls never cache; everywhere else the root session reads its history from cache from the third turn on.

7.6 Claude Opus 5.5 on coding, memo-rubric and logbook-hard

The three harder packs Opus had not run yet, in all three modes, with the routing of the September run (`root` on Opus 5.5 at high effort, `worker` and `judge` on Sonnet 5.5, `proxy` on Haiku 4.5), one fresh store per pack and mode, the conversation cached and prompt functions batched by default: coding and memo-rubric on 6 October 2026 and `logbook-hard`, capped at ten tasks like the Haiku and Luna runs, on 10 October from the same router file (Table 8). The counted runs cost $25.08, $17.70 of it on logbook-hard and $14.82 of that on its plain run; the gate's replays (26 candidates, 13 adopted) another $10.63, $7.67 of it on coding alone, so a learning run of coding costs five times what its rows show, while on logbook-hard the gate added $1.41 (10 candidates, 5 adopted).

Coding is saturated for Opus, as the original packs were. Every task in every mode, where Luna solved 11, 10 and 9 and Haiku 2, 2 and 4. The three modes cost the same within a few cents, so the pack no longer separates modes at this model size.

Memo-rubric is the one pack where Opus is behind Luna. Half the tasks in frozen and plain against Luna's 8 and 11, and learning made it worse: the adopted entries pulled Opus from 10 to 4 solved at the same cost, where Luna's learning run on the same pack went from 8 to 18. The judge is Sonnet 5.5 in both runs, so the difference is in what the models write, not in how it is scored. The gate admits an entry that wins its replay sample of three; an entry that wins three memos and loses six is the failure mode the sample size allows, and this is the first run to show it.

Pack	Mode	Solved	Calls / task	Tokens / task	Dollars
coding (12)	frozen	12/12	8.58	76 533	1.31
	learning	12/12	8.00	88 984	1.70
	plain	12/12	9.42	68 156	1.32
memo-rubric (20)	frozen	10/20	5.30	40 086	1.26
	learning	4/20	5.35	44 064	1.24
	plain	10/20	1.00	5 763	0.55
logbook-hard (10)	frozen	10/10	8.50	79 296	1.45
	learning	10/10	6.70	66 997	1.43
	plain	5/10	11.90	801 803	14.82

Table 8: Claude Opus 5.5 root with Sonnet 5.5 workers and judge, 6 and 10 October 2026, one run per cell, one store per cell.

Logbook-hard is where the SQL abstraction pays for itself. Opus solves all ten tasks in frozen and learning mode at 8.5 and 6.7 root calls a task and about $0.14 each, with no worker calls at all: it reads the 30 000-token log through `ctx` and aggregates in SQL rather than extracting with `llm_json`, where Haiku made some 1 700 worker calls a task and solved one. The plain agent on the same model solves five of ten for ten times the money: three tasks ran out of their thirty turns at over $3 each, and every turn re-sends the log, so a task averages 800 000 tokens against 79 000. Frozen beats plain on this pack with every model that ran it (Luna 7/10 to 3/10, Opus 10/10 to 5/10); only Haiku, which could not finish the SQL route, is the exception.

The caching and batching changes hold at Opus prices. Learning on coding read 5.0M tokens from cache against 30 000 billed in full, and the batched prompt functions kept the worker side to a few hundred calls per run where the Haiku logbook-hard runs made tens of thousands, and Opus on logbook-hard needed none.

7.7 The hard packs on three models

The four original packs saturated for Opus in September and separated little on Haiku and Luna, so four harder versions were built in mid October (§6) and run on all three models in all three modes: Haiku 4.5 on 13 October ($29.87 of counted runs), Luna on 13 and 14 October ($9.92) and Opus 5.5 on 26 October ($34.56), each pack and mode in its own store, with the routing of the earlier sweeps. The Opus run was the first with a per-task dollar cap, $2.50 in every mode; two tasks of the plain agent on oolong-hard hit it and count as unsolved. Luna's root ran at reasoning effort `none`, because its chat endpoint rejects native tool definitions at any other setting and the plain agent needs them. Table 9 gives every cell; Figure 5 plots pass rate against cost.

The hard packs separate models, not modes at Opus. Opus solves 198 of 204 task runs (every task of finance-hard in every mode, every frozen task of all four packs); Haiku solves between 30% and 95% of a pack and Luna between 0% and 62%. Within a model the modes are within a few tasks of each other on every pack but one, so what the hard packs measure is mostly the model, and the difference the abstraction makes shows up in cost.

Where the context must be aggregated, plain pays and loses. oolong-hard is the pack built around joins across a 25 000-token tracker, and it is the one pack where the plain agent falls behind on the strongest model: 16/20 for $13.91 against 20/20 for $2.29 frozen and $1.73 learning, a six-fold difference in cost on top of four lost tasks, two of them to the dollar cap. The

		Claude Opus 5.5			Claude Haiku 4.5			GPT-6 Luna		
Pack	Mode	Solved	Calls	$	Solved	Calls	$	Solved	Calls	$
oolong-hard (20)	learning	20/20	4.8	1.73	7/20	138.1	2.85	7/20	1 156.0	2.64
	frozen	20/20	6.8	2.29	6/20	20.9	2.45	9/20	2 651.6	5.47
	plain	16/20	5.5	13.91	6/20	20.8	7.36	0/20	1.4	0.07
finance-hard (20)	learning	20/20	5.2	1.72	15/20	9.1	1.55	8/20	88.3	0.22
	frozen	20/20	6.1	1.61	13/20	14.7	2.87	8/20	447.2	0.82
	plain	20/20	1.2	3.44	19/20	15.7	3.93	6/20	1.1	0.04
legal-hard (20)	learning	19/20	15.6	3.46	7/20	29.1	3.20	7/20	16.4	0.06
	frozen	20/20	16.6	3.26	10/20	26.9	3.13	3/20	16.1	0.05
	plain	20/20	1.1	1.88	11/20	4.6	1.26	4/20	1.5	0.02
terminal-hard (8)	learning	7/8	4.5	0.44	7/8	58.0	0.48	4/8	1 877.1	0.48
	frozen	8/8	4.5	0.37	5/8	25.2	0.43	4/8	3.2	0.05
	plain	8/8	7.6	0.45	7/8	16.9	0.36	5/8	5.9	0.00

Table 9: The four hard packs, October 2026, one run per cell, one store per cell. Calls are per task, dollars are the pack total of the solver's own calls. Opus ran under a $2.50 per-task cap; the gate's replays are not included.

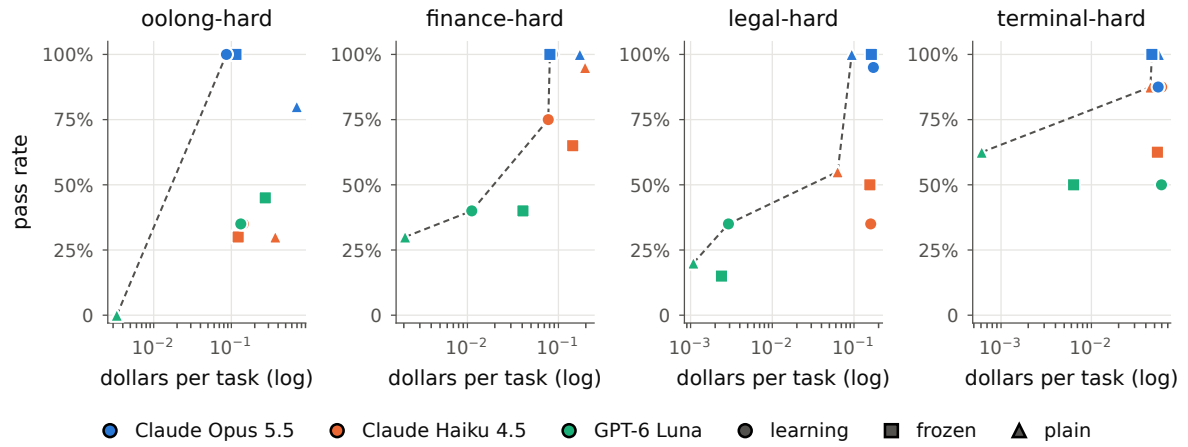


Figure 5: Pass rate against mean dollars per task (log scale) on the four hard packs, every model and mode. Colour is the model, shape is the mode; the dashed line joins the points no other point beats on both axes. On every pack the frontier runs from a cheap Luna point to an Opus KLEENE point, and the Opus plain agent sits to the right of the Opus frozen point wherever the context has to be aggregated.

plain agent re-reads the context into every turn (302 000 tokens a task against 68 000) where the SQL modes read it through `ctx` once. On Luna the plain agent solves none of the twenty for seven cents; Luna frozen solves nine at 2 650 calls a task, the per-row extraction storm of §7.4 again, this time with a join on top.

Where the question is a lookup, plain is still cheapest. On legal-hard, where the answer is a set of categories read as amended, the Opus plain agent is perfect at 1.1 calls a task and $1.88, half the SQL modes' cost; on finance-hard it is perfect too but costs twice as much, because it reads 45 000 tokens a task where the SQL modes filter the statement first. Haiku's plain agent on finance-hard solves 19/20 by brute force, 560 000 tokens a task of re-reading, so that pack separates on cost rather than accuracy for Haiku.

Learning moved little here. Opus adopted entries on every pack and cut calls per task on oolong-hard (6.8 to 4.8) and finance-hard (6.1 to 5.2) at a lower total cost, but lost a task each on legal-hard and terminal-hard. Haiku learning won on terminal-hard (7/8 against 5/8) and finance-hard (15 against 13) and lost on legal-hard (7 against 10). Luna learning is the mixed case: better on legal-hard (7 against 3), cheaper on oolong-hard and finance-hard at a fifth of the calls, and a call storm on terminal-hard (1 877 calls a task for the same four solves). With one run per cell, none of these gaps is more than the run-to-run variance a sample of twenty allows.

7.8 Across models

Table 10 collects the pass rates of every cell, and Figure 6 plots pass rate against cost per task for the three original packs that separate on accuracy (Figure 5 does the same for the hard packs). Reading across:

Pack	n	Claude Opus 5.5	Claude Haiku 4.5	GPT-6 Luna
terminal	6	100 / 100 / 100	100 / 100 / 100	83 / **100** / **100**
oolong-like	20	100 / 100 / 100	95 / 70 / **100**	**100** / 75 / 70
finance-synthetic	20	100 / 100 / 100	95 / **100** / **100**	75 / 70 / **100**
legal-synthetic	20	100 / 100 / 100	**85** / 70 / 75	75 / 90 / **100**
coding	12	100 / 100 / 100	17 / 17 / **33**	83 / **92** / 75
memo-rubric	20	20 / **50** / **50**	**55** / 40 / 40	**90** / 40 / 55
logbook-hard	10	**100** / **100** / 50	10 / 10 / **20**	33 / **70** / 30
oolong-hard	20	**100** / **100** / 80	**35** / 30 / 30	35 / **45** / 0
finance-hard	20	100 / 100 / 100	75 / 65 / **95**	**40** / **40** / 30
legal-hard	20	95 / **100** / **100**	35 / 50 / **55**	**35** / 15 / 20
terminal-hard	8	88 / **100** / **100**	**88** / 62 / **88**	50 / 50 / **62**
recursion-depth	20	– / **100** / –	– / 35 / –	– / 20 / –

Table 10: Pass rate in percent, one column per model; each cell reads learning / frozen / plain with the best mode in bold. The Haiku coding frozen and learning cells and all Haiku memo-rubric cells are from the 1 October sweep (shared memo, text-protocol baseline); every other cell ran in its own store. The Luna logbook-hard learning cell is 1 of 3 tasks. The recursion-depth row is the plain pack in frozen mode only (§7.9; the Luna cell is the rerun on the fixed cost model).

- **Where the task is an aggregation or a draft, learning wins.** On oolong-like and memo-rubric the learning mode is the most accurate mode for Luna and for Haiku, and on oolong-like it is also the cheapest KLEENE mode by an order of magnitude in calls. The adopted entries carry the shape of the answer (the grouping query, the memo's structure), so the stream becomes lookups.
- **Where the task is a lookup, plain wins.** On finance and legal the one-call agent is perfect for Luna and Opus and within a task of it for Haiku, at a tenth to a hundredth of the cost. The SQL abstraction pays for a catalog, a planner and rendered results on every turn, and a lookup does not need them.
- **Where the context does not fit, frozen wins, and on the strongest model it is also cheaper.** logbook-hard is the pack built for the claim of §1: Luna frozen solves 7/10 where the plain agent solves 3/10, Haiku frozen solves 1/10 at 1 700 calls a task, and Opus frozen solves 10/10 at $0.14 a task where the Opus plain agent solves 5/10 at $1.48. The abstraction gets the answer when the model can write the extraction, and Opus writes it as SQL over `ctx` with no worker calls at all. oolong-hard repeats the pattern with a join in the question: Opus 20/20 for $2.29 against 16/20 for $13.91.

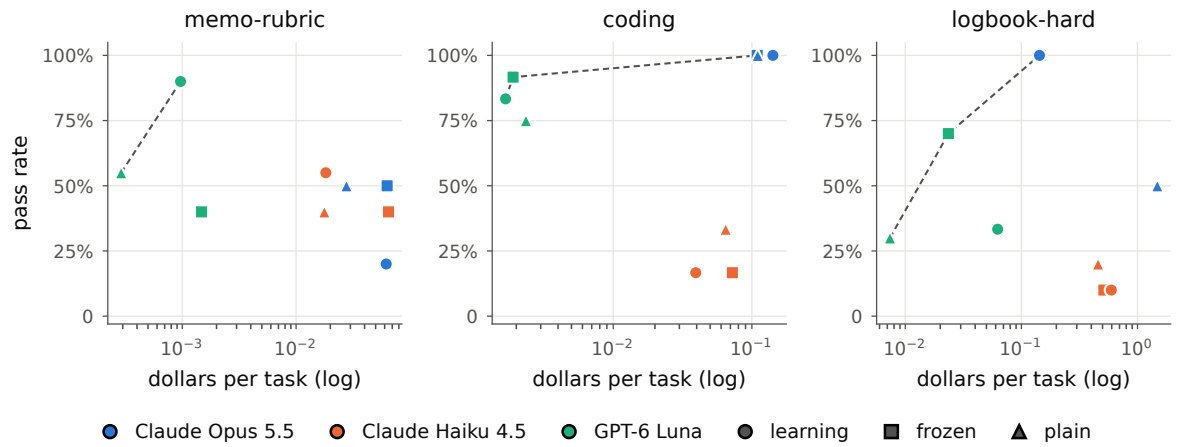


Figure 6: Pass rate against mean dollars per task of the solver's own calls (log scale), every model and mode, for the three original packs that separate on accuracy. Colour is the model, shape is the mode; the dashed line joins the points no other point beats on both axes. Gate replays and judge calls are not in the cost. Luna's learning run of logbook-hard (1 of 3 tasks) is plotted as recorded; the Opus plain point on logbook-hard sits alone at the right, $1.48 a task for half the solves.

- **Stronger models erase the modes.** Opus solves every coding step in every mode at the same price, as it solved the original packs, and 198 of 204 runs of the hard packs built to be harder. A pack that separates modes at one model size is saturated at the next, so the benchmark set has to grow with the models; the hard packs separate *models* by thirty to a hundred points and modes by a task or two.
- **The gate is still the cost of learning.** Across the sweeps the replays cost between 40% and 125% of the counted runs, and on Opus coding five times.

7.9 Recursion depth

Every pack above is flat: `evals.depth`, the deepest child session a task opened, is 0 in nearly every cell of Table 10 and 1 or 2 in a handful, so none of them says how the harness or the models behave when sessions nest, which is the premise of the design. Three packs were built for that question in late October. The generator `nest` grows *deeper* as its dial rises where every other generator grows wider: a group's unit reports, one per `ctx` row and shuffled, each naming the unit, the unit it reports to, its direct spend, distractor figures and its sub-units, with some reports excluding a divested subtree and `Correction:` rows restating a spend. The consolidated spend of a unit is its own spend plus that of its sub-units, recursively, so the answer is a recursion exactly as deep as the tree: five tasks at each of 3, 4, 5 and 6 levels (about 20, 40, 75 and 140 units; 3 000 to 25 000 characters), asking for a consolidated figure, the largest sub-unit, a count of units or the depth. Nothing forces a model to recurse, since a recursive CTE over the parsed reports answers too, so a run records the depth the model chose next to whether it was right and what it spent. Two variants keep the same trees, seeds, questions and answers: `recursion-depth-long` wraps every report in at least 10 000 characters of management commentary with figures that are not the direct spend, so a six-level group is about a million characters (260 000 tokens) and no single `llm()` call reads it; `recursion-depth-guided` adds one sentence to the task, that a sub-unit's consolidated spend is this same question over that sub-unit's reports. The guided pack asks whether a model follows a recursion it is told about, the plain pack whether it finds one, the long pack whether length alone makes it partition, and whether the partition follows the tree. An offline solver, `NestSolver`, delegates every sub-unit

through `rlm` with the same CallSQL at every level, reaches exactly the dialled depth on every task and passes every oracle; it is the integration test, not a result.

Four harness bugs the pack found first. The harness had never run a session six levels deep before, and the first runs found what the flat packs could not. A `FINAL` in the same reply as the `CREATE TABLE` it read from was planned before the table existed and refused; statements are now planned when their turn comes. Child sessions nested on the thread stack of their parent, so a deep tree deepened the stack with it; children now run as their own tasks. A `WITH RECURSIVE ... UNION ALL` with no bound grew to 14 GB inside the recursion loop before the operating system killed the process; the fixpoint now stops at a million rows with an error the model reads. And the cost model priced every call at a fixed $2 and $10 per million tokens whatever the model, so on Luna, forty times cheaper, every `rlm` over the report rows was estimated at twenty times its true cost and refused by the budget gate: on one task Luna wrote the lateral delegation three times and was told each time that $4.68 exceeded the $1 that remained, when the statement would have cost about $0.25. The gate built to keep recursion affordable was what blocked it. Rates are now learned from the trace (§3.1), and the rerun below is on the fixed binary.

Model	Pack	Solved	Depth (max)	Tasks > 0	Children	Calls / task	Dollars
Haiku 4.5	plain	7/20	0 (0)	0	0	20.1	3.66
Luna, fixed rate	plain	5/20	0 (0)	0	0	109.2	0.19
Luna, learned rate	plain	4/20	0.25 (2)	4	99	326.1	0.44
Luna	guided	3/20	0.05 (1)	1	64	124.2	0.18
Luna	long	2/20	0.90 (5)	5	1 634	1 139.2	3.26
Opus 5.5	plain	20/20	0 (0)	0	0	6.75	1.85

Table 11: The recursion-depth packs, frozen mode, 29 October to 2 November 2026, one run per cell, a $1 per-task cap ($2 for Opus). Depth is the mean over tasks of the deepest child session, with the maximum in parentheses; Children is the number of child sessions the run opened.

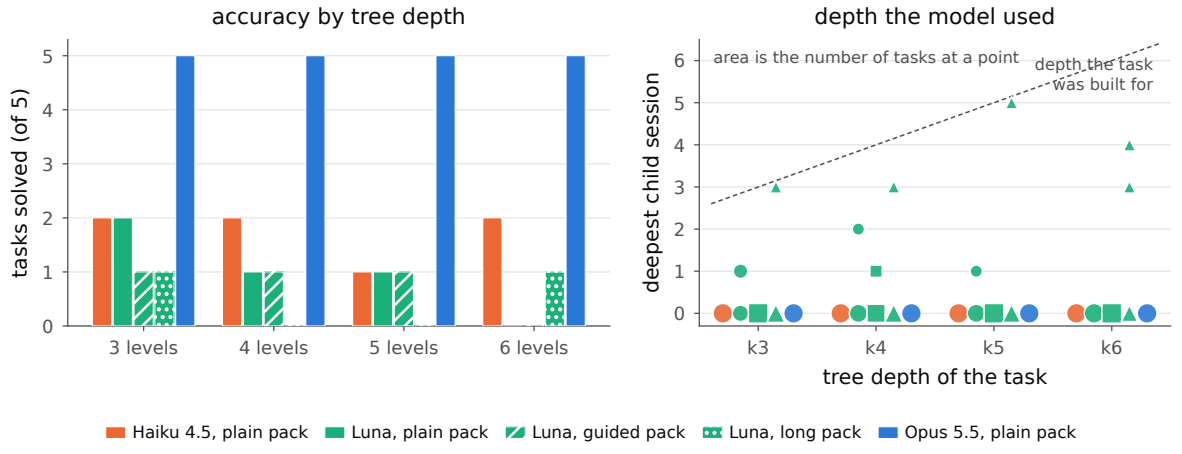


Figure 7: The recursion-depth runs. Left: tasks solved out of five at each tree depth, one bar per run. Right: the deepest child session each task opened against the depth the task was built for; the dashed diagonal is a solver that recurses along the tree, which only the offline `NestSolver` does. Luna on the long pack is the only run that leaves the floor, and it reaches depth 5 on the one task built for it.

No model recursed along the tree. Haiku never delegated: every task was flat SQL over `ctx`, right on the structural questions (how many levels, how many units) and wrong on the consolidated figures, where corrections and excluded subtrees have to be combined (Figure 7). Luna, once the gate let it, opened 99 child sessions over four tasks, depth 1 on three and depth 2 on one it then solved exactly; but the shape is a map, not a divide: every child gets one report row and the question about the top unit, so the children extract and the root still has to consolidate. Nine tasks hit the thirty-turn cap, mostly at five and six levels. The guided sentence did nothing: half the tasks ended on turn one with a guessed `FINAL SELECT 0`, and the one task that delegated went one level.

Length moved the depth; nothing yet moved the shape. On the long pack Luna went to depth 3 on three tasks, depth 4 on one and depth 5 on `k5-05`, the depth that task was built for, through 1 634 child sessions (653 at depth 1, 652 at depth 2, 317 at depth 3, 9 at depth 4, 3 at depth 5), every one of which finished, so the harness side of the question has an answer at five levels. The model side does not: the recursion is text partitioning, children extracting facts from chunks of a long report and delegating chunks of their chunk, not consolidation along the tree, and none of it made the answer better (2/20, nine tasks out of turns, $0.65 on a single six-level task, two and a half hours of wall clock for the pack).

Opus answers the recursion in SQL. 20/20 for $1.85 at 6.75 calls a task, depth 0 on every task and no `llm()` at all: Opus parses the reports with string functions and consolidates with a `WITH RECURSIVE` CTE, 25 of them over the run, one plan at every depth from 3 to 6. That is the flat solution the pack always allowed, done right, and it says what the recursion packs measure: depth is interesting only for a model that cannot do this, and a model that can turns the tree into a fixpoint the engine computes. Whether Opus recurses when the context no longer fits one statement, on the long pack, is the run that has not been made (§10).

7.10 Planner estimates against actuals

The Haiku run was the first made after `bench run` began tracing statements, so its store holds an estimate and an actual call count for each of the 1 028 statements the planner priced (Figure 8). Reading the plot: 670 statements made no calls and were estimated at none; 254 were estimated at one call and made one; 82 were estimated at one and made none, which is the memo answering a repeated prompt. The planner never under-estimated by more than a call. Where it was wrong it was high, by up to two orders of magnitude on about a dozen statements, each a per-row call priced on a table's row count that then failed early or was served from the memo. For budget refusal that is the safe direction; for the learned cost model it is the error to drive down. The later sweeps traced 4 517 (Luna), 2 755 (Haiku, 6 October) and 928 (Opus, 6 October) further statements, with the same shape.

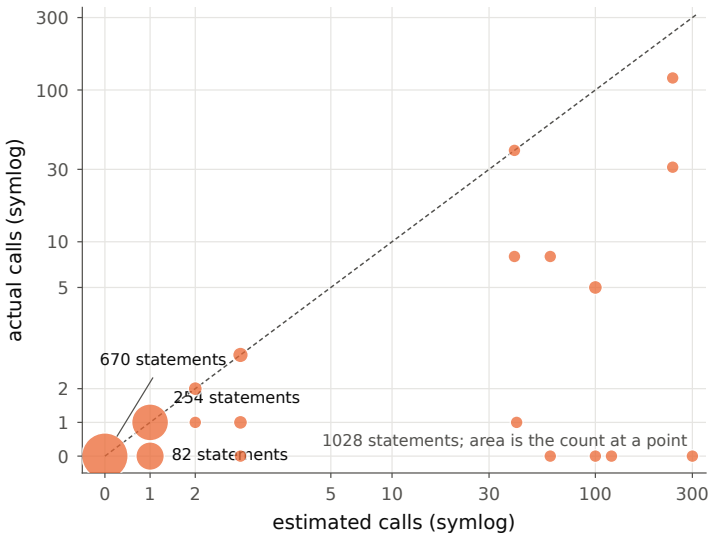


Figure 8: Estimated against actual calls per statement, the 1 October Haiku run, on symmetric log axes; the area of a point is the number of statements at it. Points on the diagonal are exact estimates; every outlier lies below it, an over-estimate.

7.11 What the numbers say

Taken together, the sweeps support five statements and refuse a sixth. First, the planner, the budget and the trace work as designed on real models of three families: estimates are conservative, refusals carry plans, per-row calls land on the cheap alias, and every cost in this section was read from the store; the one time the cost model was wrong in the other direction, a fixed rate on a cheap model, the trace showed it and the rate is now learned. Second, the playbook reduces calls per task when something is adopted, and on the packs whose tasks share a shape (aggregation, drafting against a rubric) it also raises solves, for two of the three models. Third, the replay gate, at its current sample size, is the dominant cost of learning at pack scale, and a sample of three can admit an entry that hurts (Opus on memo-rubric). Fourth, the long-context case the design was built for is now measured on all three models: on the two packs whose context has to be aggregated rather than looked up, KLEENE frozen is the most accurate mode for every model that could write the extraction, and on the strongest model it is also four to ten times cheaper than the agent that re-reads the context every turn. Fifth, the harness recurses to five levels without losing a child, but no model yet recurses along the structure of the task: the strongest turns the tree into a recursive CTE, the cheapest partition text. What the runs still do not support is a general cost-parity claim: wherever the context fits in one prompt and the question is a lookup, the plain agent is as accurate and far cheaper, and the strongest model saturates most of the packs, the hard ones included.

8 Related work

Recursive language models. Zhang et al. [42] introduced the REPL-variable framing and the `llm_query` / `rlm_query` sub-calls that KLEENE maps onto `llm` and `rlm`; their evaluation suite (OOLONG, BrowseComp-Plus) is the headline comparison the `logbook-hard` pack is built toward. Wang [39] reproduced the result and documented the depth-two degradation that set KLEENE's default depth cap. DSPy's `dspy.RLM` [18] is the most used implementation; like the reference harness it is Python-based with fixed iteration and call caps rather than a priced plan. CodeAct [40] made the general case that executable code is a better action space than JSON

tool calls; KLEENE agrees and narrows the language to one with an optimizer.

SQL and relational operators over language models. The closest systems embed model calls in a relational language and optimise them. LOTUS [29] defines semantic filter, map, join, aggregate and top-k operators with proxy cascades and statistical guarantees; KLEENE's cascade rule has LOTUS's shape with thresholds declared rather than calibrated. Palimpzest [23] and Abacus [31] run a Cascades-style [11] cost-based optimiser over model and prompt choices using sampled sentinel plans; KLEENE's planner is simpler (eight rules, one cost model) but operates on the model's own SQL at run time rather than on a pre-declared pipeline. ThalamusDB [17] adds `NLfilter` and `NLjoin` to DuckDB with progressive error bounds and call budgets. BlendSQL [10] is closest in spirit to CallSQL: a dialect with `LLMMap`, `LLMQA` and `LLMJoin` whose AST is rewritten so cheap SQL predicates run first; SUQL [24] exposes free-text primitives as lazily evaluated Postgres UDFs; Galois [32] treats the model itself as the database executing plan fragments; CAESURA [37] uses the model as the planner; Evaporate [2] synthesises extraction code instead of prompting per row, which is what a refined `CREATE FUNCTION ... AS SQL` amounts to. Liu et al. [25] reorder rows to share prompt prefixes in relational workloads, a rule KLEENE does not yet have. Scallop [22] makes foundation models foreign predicates in Datalog. Su et al. [36] give a taxonomy and benchmark of model-enhanced relational operators. What distinguishes KLEENE from all of these is that the *model writes the query*, turn by turn, with delegation, tools, budgets and learning in the same language, rather than a person writing a pipeline the model fills in.

Query optimisation. The planner's machinery is classical: System R's dynamic programming [33], bushy enumeration without cross products [28], semi-naive fixpoint evaluation [3], and the complexity results of Chandra and Merlin [6], Vardi [38], Aho and Ullman [1] and Dantsin et al. [8] that label each statement's fragment. The name honours three of Kleene's results [19–21]: the star as closure under repetition (what a recursive CTE computes), the fixed-point construction (how it computes it), and the recursion theorem (a session opening a child session is the same loop referring to itself).

Continual harnesses and curricula. The loop's learnability target follows Absolute Zero Reasoner [43]; its generators follow Reasoning Gym [35] and the 3-SAT phase transition [27]; its ratings are Bradley–Terry [5]. The replay gate is the harness-level analogue of the default-FAIL test contract common to long-running agent harnesses: nothing is adopted that has not beaten the current version on held tasks.

9 Key takeaways

1. **SQL is a workable action language for an agent, and it buys a planner.** Three model families wrote CallSQL well enough to solve terminal, extraction, aggregation, drafting and code-editing tasks through it, and every statement they wrote was priced before it ran. Across 9 000 traced statements the planner never under-estimated by more than a call; its errors are over-estimates, the safe direction for a budget.
2. **The abstraction pays where the context does not fit a prompt or the task repeats.** On the 30 000-token log, KLEENE frozen is the most accurate mode for every model that could write the extraction (Luna 7/10 against the plain agent's 3/10; Opus 10/10 against 5/10 at a tenth of the cost), and on the hard aggregation pack Opus frozen is 20/20 for $2.29 where the plain agent is 16/20 for $13.91. On a stream of aggregation or drafting tasks the playbook turns the stream into lookups: Luna memo-rubric 8/20 to 18/20, oolong-like 15/20 to 20/20 at 1.3 calls a task against 34.

3. **It does not pay for one-call lookups.** Wherever the context fits one prompt and the question is a single fact, the native tool-calling agent is as accurate and ten to a hundred times cheaper. The catalog, the planner and the rendered results are overhead there, and the numbers say so on finance, legal and memo-rubric for every model.

4. **Learning is cheap to run and expensive to gate.** Adopted entries cut calls per task on every pack where something was adopted, but the replay gate costs 40% to 500% of the counted run at twenty tasks a pack, and a sample of three can admit an entry that hurts (Opus on memo-rubric, 10 to 4). The gate's sample size, not the playbook, is the open design question.

5. **Cost is in the worker calls, and batching is the lever.** The expensive runs were per-row extraction over long contexts: 79 000 worker calls on one Haiku pack. Batching 20 rows a call with an item-wise re-ask, conversation caching and strict-schema rewriting were each forced by a run and each cut the next run's cost; the Opus coding run read 5M tokens from cache against 30 000 billed.

6. **Benchmarks must grow with the models, and they did not grow fast enough.** Opus 5.5 saturated five of the first seven packs, so four harder ones were built; it solves 198 of 204 runs of those too. The hard packs separate models by thirty to a hundred points and modes by a task or two, so they say which model to use and, through cost, whether to put SQL between it and the context. The imported real datasets (OOLONG, Harvey LAB, contract redlining) remain the next step.

7. **Depth is a harness property now, not yet a model behaviour.** Child sessions nest five deep and 1 634 of them finish, the budget gate no longer prices a cheap model out of delegating, and a recursive CTE that runs away is stopped. But no model recursed along the tree the task was built on: Opus solves every recursion-depth task flat with a recursive CTE, Haiku never delegates, and Luna delegates only when the context is too long to read, partitioning text rather than consolidating units. Making a model recurse along structure, where structure is the cheaper route, is the open problem the packs now measure.

10 Limitations and future work

- **One run per cell.** Every number in §7 is a single run. On twenty tasks accuracy moves in steps of five points, and a difference of one task is not a result. Repeated runs are the first thing to add.

- **Saturated and synthetic packs.** Opus 5.5 solves nearly every task of ten of the eleven packs, the four built to be harder included, and the synthetic packs are generated, so a solver that learns the generator's shape will look better than one meeting the real datasets. Importers exist for the real OOLONG (from Hugging Face), Harvey LAB and two contract-redlining datasets, but no model has run them yet.

- **The recursion-depth packs have run in frozen mode only, and not every pair.** Opus on the long pack, where the context no longer fits one statement, is the run that would say whether the strongest model recurses when it has to; it has not been made (an estimated $15 to $25). Luna's root ran at reasoning effort `none` for its plain agent's sake, and half its guided tasks ended on turn one with a guess; a frozen rerun at a low effort needs no native tools and is cheap. A learning run, so that one task's working `rlm` plan reaches the next, has not been tried on any of the three.

- **The first Haiku sweep is not like the others.** Its coding and memo-rubric cells share a memo between frozen and learning, use the text protocol baseline, and predate the finish

check and the schema rewrite. They are kept because they are what was measured; a rerun would replace them.

- **The gate's cost and its sample.** Six task runs per solved task dominates a learning run at pack scale, and a sample of three admitted entries that hurt Opus on memo-rubric. A larger or adaptive sample, a proxy model for the replay, and a cached baseline arm are the candidates.
- **Capped tasks are unsolved tasks.** The Opus hard-pack run and every recursion run had a per-task dollar cap, and a task that hit it counts as a failure with its reason recorded. Two of the Opus plain agent's oolong-hard tasks and several Luna long-pack tasks are such rows; a higher cap might have solved them at a price the table does not show.
- **Judge calls are untraced.** The Sonnet judge's cost is not in any table; it is one short call per memo task, roughly a dollar a sweep.
- **Declared thresholds, per-session selectivities.** Cascade thresholds are declared, not calibrated from a sample as in LOTUS; sampled selectivities persist only within a session rather than as versioned tables. Token rates are learned from the trace, but only once three priced calls exist, so the first statements of a fresh store are priced at the default.
- **Schema rewriting is partial.** Model-written `llm_json` schemas are rewritten into each provider's accepted subset only for the shapes seen so far.

Beyond these, the design names measurements the harness supports but no run has yet exercised: transfer (learn on a prefix with `-limit`, run the rest frozen on the same store, compare against a never-learned store), reverts through `learn playbook`, and the plan-space plot over the shapes models actually write.

11 Reproducibility

Kleene v0.5.0 (29 October 2026) ships prebuilt binaries for macOS, Linux and Windows (a zip of `kleene.exe`; v0.1.0 of 6 October was the first release); the repository is its own Homebrew tap, and releases are cut by release-please from the conventional-commit history:

```
curl -fsSL https://raw.githubusercontent.com/MarcusElwin/kleene/main/install.sh | sh
brew tap MarcusElwin/kleene https://github.com/MarcusElwin/kleene && brew install
    kleene
```

The repository is a Rust workspace; from source, DuckDB builds once (about ten minutes). Every claim in Table 3 is a test:

```
cargo test --workspace --all-features
cargo run -- repl < demos/planner/three_way.sql     # the plan space, live
cargo run -- learn run --tasks 20 --generators puzzle,corpus
cargo run -- bench run tasks/terminal --mode frozen --record fixtures/terminal
cargo run -- bench report
cargo run -- bench plot plots/
```

The per-task rows of every sweep are under `plots/`, each directory with the `bench report` output, the router file and, where `bench plot` ran, the SVGs beside it:

- `evals.csv` for §7.2;
- `haiku-2026-10-01`, `luna-2026-10-05`, `haiku-2026-10-06` and `opus-2026-10-06` (with its `logbook-hard` subdirectory) for §7.3 to §7.6;
- `haiku-2026-10-13`, `luna-2026-10-13` and `opus-2026-10-26` for the hard packs (§7.7);

- `haiku-recursion-2026-10-29`, `luna-recursion-2026-10-31` (with `rerun`, `guided` and `long`) and `opus-recursion-2026-11-02` for §7.9.

`kleene bench results` regenerates Table 10, Figure 6 and Figure 5 from those CSVs, and `paper/figures/make_figures.py` draws every figure in this report from the same rows. The model replies of the first Opus run were recorded but are not checked in, because the catalog bug of §7.2 means they cannot replay the run; later runs record with `-record` and replay with `-replay`. Prices are the router's list rates at the time of each run, per million input / output tokens: $4 / $20 for Opus 5.5, $2 / $10 for Sonnet 5.5, $1 / $5 for Haiku 4.5, $0.10 / $0.50 for GPT-6 Luna.

Acknowledgements. The implementation, the benchmark runs and much of the documentation were produced with coding agents working from the plan in `docs/PLAN.md`; the design decisions, the measurements and the mistakes are the author's.

References

[1] Alfred V. Aho and Jeffrey D. Ullman. Universality of data retrieval languages. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 110–119, 1979. doi: 10.1145/567752.567763.

[2] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes. *arXiv preprint arXiv:2304.09433*, 2023. URL `https://arxiv.org/abs/2304.09433`.

[3] François Bancilhon and Raghu Ramakrishnan. An amateur's introduction to recursive query processing strategies. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*, pages 16–52, 1986. doi: 10.1145/16894.16859.

[4] Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, and Graham Neubig. Oolong: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025. URL `https://arxiv.org/abs/2511.02817`.

[5] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.2307/2334029.

[6] Ashok K. Chandra and Philip M. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of the 9th Annual ACM Symposium on Theory of Computing (STOC)*, pages 77–90, 1977. doi: 10.1145/800105.803397.

[7] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169, 2000. doi: 10.1007/10722167_15.

[8] Evgeny Dantsin, Thomas Eiter, Georg Gottlob, and Andrei Voronkov. Complexity and expressive power of logic programming. *ACM Computing Surveys*, 33(3):374–425, 2001. doi: 10.1145/502807.502810.

[9] Andrew Gierth. Cyclic tag system. PostgreSQL wiki, `https://wiki.postgresql.org/wiki/Cyclic_Tag_System`, 2010. A Turing-complete construction in SQL:2003 with `WITH RECURSIVE`.

[10] Parker Glenn, Parag Pravin Dakle, Liang Wang, and Preethi Raghavan. BlendSQL: A scalable dialect for unifying hybrid question answering in relational algebra. *arXiv preprint arXiv:2402.17882*, 2024. URL https://arxiv.org/abs/2402.17882.

[11] Goetz Graefe. The cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.

[12] Harvey AI. Harvey LAB: the legal agent benchmark. https://github.com/harveyai/harvey-labs, 2026. Repository and evaluator; accessed 2026-10-03.

[13] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. CUAD: An expert-annotated NLP dataset for legal contract review. In *NeurIPS Datasets and Benchmarks Track*, 2021. URL https://arxiv.org/abs/2103.06268.

[14] Toshihide Ibaraki and Tiko Kameda. On the optimal nesting order for computing N-relational joins. *ACM Transactions on Database Systems*, 9(3):482–502, 1984. doi: 10.1145/1270.1498.

[15] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023. URL https://arxiv.org/abs/2311.11944.

[16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

[17] Saehan Jo and Immanuel Trummer. ThalamusDB: Approximate query processing on multi-modal data. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654989.

[18] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023. URL https://arxiv.org/abs/2310.03714.

[19] Stephen C. Kleene. On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4): 150–155, 1938. doi: 10.2307/2267778.

[20] Stephen C. Kleene. *Introduction to Metamathematics*. North-Holland, 1952.

[21] Stephen C. Kleene. Representation of events in nerve nets and finite automata. In Claude E. Shannon and John McCarthy, editors, *Automata Studies*, pages 3–41. Princeton University Press, 1956.

[22] Ziyang Li, Jiani Huang, and Mayur Naik. Scallop: A language for neurosymbolic programming. *Proceedings of the ACM on Programming Languages*, 7(PLDI), 2023. doi: 10.1145/3591280.

[23] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. A declarative system for optimizing AI workloads. In *Conference on Innovative Data Systems Research (CIDR)*, 2025. URL https://arxiv.org/abs/2405.14696. arXiv:2405.14696.

[24] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. SUQL: Conversational search over structured and unstructured data with large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. URL https://arxiv.org/abs/2311.09818.

[25] Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Mo, Lei Cao, and Ion Stoica. Optimizing LLM queries in relational data analytics workloads. *arXiv preprint arXiv:2403.05821*, 2024. URL https://arxiv.org/abs/2403.05821.

[26] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. URL https://arxiv.org/abs/2601.11868.

[27] David Mitchell, Bart Selman, and Hector Levesque. Hard and easy distributions of SAT problems. In *Proceedings of the 10th National Conference on Artificial Intelligence (AAAI)*, pages 459–465, 1992.

[28] Guido Moerkotte and Thomas Neumann. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*, pages 930–941, 2006.

[29] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators: A declarative model for rich, AI-based data processing. In *Proceedings of the VLDB Endowment*, 2025. URL https://arxiv.org/abs/2407.11418. arXiv:2407.11418.

[30] Mark Raasveldt and Hannes Mühleisen. DuckDB: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984, 2019. doi: 10.1145/3299869.3320212.

[31] Matthew Russo, Chunwei Liu, Sivaprasad Sudhir, Gerardo Vitagliano, Samuel Madden, Michael Cafarella, and Michael Franklin. Abacus: A cost-based optimizer for semantic operator systems. *arXiv preprint arXiv:2505.14661*, 2025. URL https://arxiv.org/abs/2505.14661.

[32] Mohammed Saeed, Nicola De Cao, and Paolo Papotti. Querying large language models with SQL. *arXiv preprint arXiv:2304.00472*, 2023. URL https://arxiv.org/abs/2304.00472.

[33] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979. doi: 10.1145/582095.582099.

[34] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[35] Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, et al. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2505.24760*, 2025. URL https://arxiv.org/abs/2505.24760.

[36] Yunxiang Su, Tianjing Zeng, Zhongjun Ding, Yin Lin, et al. Large language model-enhanced relational operators: Taxonomy, benchmark, and analysis. *arXiv preprint arXiv:2603.02537*, 2026. URL https://arxiv.org/abs/2603.02537.

[37] Matthias Urban and Carsten Binnig. CAESURA: Language models as multi-modal query planners. *arXiv preprint arXiv:2308.03424*, 2023. URL https://arxiv.org/abs/2308.03424.

[38] Moshe Y. Vardi. The complexity of relational query languages. In *Proceedings of the 14th Annual ACM Symposium on Theory of Computing (STOC)*, pages 137–146, 1982. doi: 10.1145/800070.802186.

[39] Daren Wang. Think, but don't overthink: Reproducing recursive language models. *arXiv preprint arXiv:2603.02615*, 2026. URL https://arxiv.org/abs/2603.02615.

[40] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.01030.

[41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

[42] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025. URL https://arxiv.org/abs/2512.24601.

[43] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025. URL https://arxiv.org/abs/2505.03335.